Fin-Comply â€" DPDP Compliance System: Complete Technical Explanation

> **Audience:** Engineering evaluators & non-technical compliance officers
> **Scope:** Pure theory â€" no code changes
> **Codebase:** `compliance_sys/` (Python/FastAPI backend + React/Vite frontend)

PART 1 â€" SYSTEM OVERVIEW

What Does Fin-Comply Actually Do?

Fin-Comply is an automated compliance checking system for India's **Digital Personal Data Protection (DPDP) Act, 2023**. It answers one question for any fintech company: "*Are we following the law when we handle people's personal data?*"

Think of it like a fire safety inspector for data privacy. Just as a fire inspector walks through a building checking extinguishers, exits, and alarms against a checklist of regulations â€" Fin-Comply walks through a company's data records and checks them against 14 specific obligations from the DPDP Act.

The Problem It Solves

Before Fin-Comply, a compliance officer at a fintech company would have to:

1. Manually read every consent record to check if consent was collected before processing
2. Cross-reference transaction logs against consent expiry dates
3. Check if breach notifications were sent within 72 hours
4. Verify that children's data had guardian consent
5. Confirm access logs show only authorized roles viewing PII

For a company processing thousands of transactions, this manual work is **impossible to do accurately**. A single missed expired consent record could mean a â‚¹250 crore penalty. Fin-Comply automates this entire process in seconds.

Who Uses It

The end user is a **compliance officer at a fintech company** (such as a lending platform, payment processor, or digital bank). They are responsible for ensuring their company follows the DPDP Act.

What They Upload and What They Get Back

Input: The officer uploads **10 CSV files** through a web onboarding page. These files contain the company's consent records, transaction logs, access logs, security events, data lifecycle records, DSAR requests, system inventory, policies, customer master data, and governance configuration.

Output: They receive:

- A **risk score** from 0 to 100 (e.g., 60.1 = HIGH risk)

- A **list of every violation** found, grouped by DPDP rule
- A **SHAP-based explanation** of why each violation was flagged
- A **remediation priority matrix** telling them what to fix first
- A **downloadable PDF report** for board presentations

Why This Matters Under the DPDP Act

The DPDP Act 2023 imposes penalties up to **â‚¹250 crore** for violations. Companies must demonstrate compliance, not just claim it. Fin-Comply provides auditable, explainable, evidence-backed compliance assessments that can be presented to the Data Protection Board of India.

The End-to-End Journey in One Paragraph

A compliance officer uploads 10 CSV files on the React onboarding page; the FastAPI backend receives them and runs `csv_loader.py` to validate structure, then `field_mapper.py` to anonymize all sensitive fields using HMAC-SHA256 hashing, timestamp truncation, bucketing, suppression, and ENUM validation; `db_writer.py` writes the anonymized data into PostgreSQL in foreign-key order; the orchestrator (`orchestrator.py`) opens a database connection and dispatches three specialized agents â€" Regulation, Audit, and Risk â€" that query the database and evaluate 14 DPDP rules using weighted signal scoring; violations are collected and fed into `compute_risk_score()` which uses a Likelihood Ã— Impact formula to produce a 0â€"100 risk score; the explainability service (`service.py`) enriches each violation with SHAP-equivalent signal attribution tables, root cause categories, and remediation steps; `report_builder.py` assembles the complete JSON report with executive summary and remediation priority; the API returns this to the React dashboard which renders six sections (executive summary, metrics, charts, violations with expandable SHAP tables, remediation matrix, and agent breakdown); and optionally, `pdf_generator.py` produces a professional ReportLab PDF for board presentation.

System Block Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                    REACT FRONTEND (Vite)                             │
│  Onboarding.jsx ──→ Dashboard.jsx ──→ PDF Download                  │
│  [Upload 10 CSVs]   [6 Dashboard Sections]    [ReportLab PDF]        │
└──────────────────────────────────────────────────────────────────────┘
                          │
                          │ HTTP POST /analyze/upload
                          ▼
┌──────────────────────────────────────────────────────────────────────┐
│                    FastAPI BACKEND (routes.py)                       │
│  Receives files → Orchestrates pipeline → Returns JSON response      │
└──────────────────────────────────────────────────────────────────────┘
                          │
                          │
          ┌───────────────┼───────────────┐
          ▼               ▼               ▼

┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│  csv_loader  │   │ field_mapper │   │  db_writer   │
│  (validate)  │──→│ (anonymize)  │──→│ (PostgreSQL) │
│  10 CSVs     │   │ 5 techniques │   │ FK-ordered   │
└──────────────┘   └──────────────┘   └──────────────┘
                                              │
                                              │ DB ready
                                              ▼
```

```
┌──────────────────────────────────────────────────────────────────────┐
│                    ORCHESTRATOR (orchestrator.py)                    │
│  Opens 1 DB connection → dispatches 3 agents → collects violations  │
└──────────────────────────────────────────────────────────────────────┘
        │                      │                      │
        ▼                      ▼                      ▼

┌──────────────┐   ┌──────────────────┐   ┌──────────────┐
│  REGULATION  │   │     AUDIT        │   │    RISK      │
│  AGENT       │   │     AGENT        │   │    AGENT     │
│  DPDP-001    │   │     DPDP-003     │   │  DPDP-006    │
│  DPDP-002    │   │     DPDP-009     │   │  DPDP-007    │
│  DPDP-004    │   │     DPDP-010     │   │  DPDP-008    │
│  DPDP-005    │   │     DPDP-011     │   │  DPDP-014    │
│              │   │     DPDP-012     │   │              │
│              │   │     DPDP-013     │   │              │
└──────────────┘   └──────────────────┘   └──────────────┘
        │                  │                    │
        └──────────────────┼────────────────────┘
                           │
                           │ List[ViolationRecord]
                           ▼

┌──────────────────────────────────────────────────────────────────────┐
│              RISK SCORING (compute_risk_score)                       │
│  L × I formula → risk_score (0-100) → tier (COMPLIANT→CRITICAL)     │
└──────────────────────────────────────────────────────────────────────┘
                           │
                           ▼

┌──────────────────────────────────────────────────────────────────────┐
│           EXPLAINABILITY (service.py + report_builder.py)            │
│  SHAP attribution → root cause → remediation → executive summary     │
└──────────────────────────────────────────────────────────────────────┘
                           │
                           │
                ┌──────────┼──────────┐
                ▼          ▼          ▼
        JSON API      React Dashboard   PDF Report
        Response      (6 sections)      (ReportLab)
```

PART 2 â€" DATA LAYER

The 10 CSV Files

Each CSV represents a real data domain within a fintech company. Together, they provide the complete picture needed to assess DPDP compliance.

1. `governance_config.csv` â€" The company's organizational setup. Contains whether a DPO (Data Protection Officer) is assigned, audit frequency, grievance mechanism availability, and risk level. This is always exactly 1 row per tenant. *DPDP obligation:* Section 13 (grievance mechanism), Section 8(4) (audit trails).

2. `customer_master.csv` â€" Every customer the company serves. Contains whether the customer is a minor, their data principal type, account status, and KYC status. The raw name, age, gender, email, phone, and address columns are **suppressed** (dropped entirely). *DPDP obligation:* Section 9 (children's data protection).

3. `consent_records.csv` â€" Every consent collected from customers. Contains consent status (active/expired/revoked), purpose, expiry date, whether notice was provided, whether consent was bundled, and the consent channel. *DPDP obligation:* Section 6(1) (valid consent), Section 5 (notice requirement).

4. `transaction_events.csv` â€" Every data processing event. Contains what happened, the purpose, whether data was shared with a third party, and whether it crossed borders. *DPDP obligation:* Section 4(1)(b) (purpose limitation), Section 6(1) (third-party sharing).

5. `access_logs.csv` â€" Every time an employee accessed customer data. Contains the employee's role, whether they accessed PII, the reason, and the outcome. *DPDP obligation:* Section 8(5) (access control).

6. `data_lifecycle.csv` â€" Tracks data retention. Contains retention expiry dates, whether the processing purpose is completed, erasure requests, and legal holds. *DPDP obligation:* Section 8(7) (retention limits), Section 8(7)(a) (purpose completion).

7. `security_events.csv` â€" Security incidents and encryption status. Contains whether PII is encrypted, breach detection, notification delays, and affected user counts. *DPDP obligation:* Section 8(5) (security safeguards), Section 8(6) (breach notification).

8. `dsar_requests.csv` â€" Data Subject Access Requests. Contains erasure requests, SLA dates, fulfillment status, and whether SLA was breached. *DPDP obligation:* Section 12(a) (right to erasure).

9. `system_inventory.csv` â€" IT systems that store personal data. Contains system type, encryption status, whether a Data Processing Agreement (DPA) is signed, and the processor type. *DPDP obligation:* Section 4(1)(c) (data minimization).

10. `policies.csv` â€" Company policies (retention periods, consent policies, etc.). Contains policy type, numeric values, and whether the policy is active. *DPDP obligation:* Supports data lifecycle and retention checks.

Why These 10 Tables

These 10 tables were chosen because they collectively cover all the evidence needed to evaluate the 14 DPDP obligations. Each table maps to one or more DPDP Act sections. Without `consent_records`, you cannot check consent validity. Without `security_events`, you cannot check breach notification timing. The selection mirrors how real GRC platforms (like OneTrust or RSA Archer) structure compliance data.

Table Relationships and FK Order

Tables reference each other through hashed identifiers. For example, `transaction_events` references `customer_master` (via `customer_hash`) and `consent_records` (via `consent_hash`). This means `customer_master` must be written to the database before `transaction_events`, otherwise the foreign key constraint fails. The insertion order in `db_writer.py` is: governance_config â†' system_inventory â†' policies â†' customer_master â†' consent_records â†' transaction_events â†' access_logs â†' data_lifecycle â†' security_events â†' dsar_requests.

Tenant Isolation

A "tenant" is a single company using Fin-Comply. Every table has a `tenant_id` column, and every query in every agent includes `WHERE tenant_id = %s`. This ensures Company A's data never leaks into Company B's

analysis. This is critical because Fin-Comply is designed as a multi-company tool â€" multiple fintechs can use it without seeing each other's compliance data. In `csv_loader.py`, tenant filtering happens at line 133: `df = df[df["tenant_id"] == tenant_id].copy()`.

PART 3 â€" ANONYMIZATION AND MASKING MODULE

Why Can't We Just Store the Raw CSV Data Directly?

Because the raw CSVs contain personally identifiable information (PII) â€" customer IDs, employee IDs, timestamps with time-of-day precision, and exact user counts. Storing these directly would mean Fin-Comply itself becomes a privacy risk. If Fin-Comply's database were breached, every customer's identity would be exposed. The anonymization module ensures that the system can check compliance *without ever storing the raw identifiers*.

Think of it like a doctor who needs to check if a patient followed their medication schedule. The doctor doesn't need to know the patient's name â€" just a unique code that links all the records together.

Technique 1 â€" HMAC-SHA256 Tokenization

What it is: Converting a readable identifier like `"CUST-4821"` into an irreversible hash like `"a3f9c12b8e01"`. The hash always produces the same output for the same input, so you can still join records across tables, but you cannot reverse the hash to recover the original ID.

Why HMAC over plain SHA256: HMAC adds a secret key to the hashing process. Plain SHA256 of `"CUST-4821"` is the same everywhere â€" an attacker could precompute hashes of common IDs (a "rainbow table") and reverse them. HMAC uses a key composed of `SALT:tenant_id`, making the hash unique to each tenant and unpredictable without the key.

Why the hash is still joinable: Because the same input always produces the same output with the same key, `customer_id = "CUST-4821"` in `consent_records.csv` produces the same `customer_hash` as in `transaction_events.csv`. The agents can JOIN these tables on `customer_hash` without ever knowing the real customer ID.

Evidence basis: NIST SP 800-107 Section 5.3 â€" keyed hash for pseudonymization of identifiers that must remain joinable.

Which columns: All `*_id` columns (customer_id, consent_id, event_id, access_id, employee_id, lifecycle_id, security_id, dsar_id, system_id, policy_id, third_party_id). Implemented in `hasher.py`.

Before/After:

```
customer_id "CUST-4821"  â†'  customer_hash "a3f9c12b8e01"
event_id    "EVT-9920"   â†'  event_hash    "7d2e44f1c903"
```

Technique 2 â€" Timestamp Truncation

What it is: Removing the time-of-day portion from timestamps, keeping only the date.

Why timestamps are dangerous: If you know that a specific transaction happened at exactly 14:23:07, and you also know that only one customer transacted at that exact second, you can re-identify the customer. By truncating to date-only, many customers share the same date, making re-identification much harder.

Evidence basis: ISO/IEC 20889:2018 Section 8.3 â€" generalisation technique for reducing precision of quasi-identifiers.

Before/After:

```
"2024-03-15 14:23:07"  â†'  "2024-03-15"
```

Implemented in: `_truncate_to_date()` in `field_mapper.py`, applied to all timestamp columns: `consent_date`, `expiry_date`, `event_date`, `access_date`, `breach_confirmed_date`, `submitted_date`, `last_audit_date`, etc.

Technique 3 â€" Bucketing

What it is: Replacing exact numeric values with categorical bands.

Why raw numbers reveal too much: If a breach affected exactly 4,821 users, that specific number might identify which breach it was. By mapping to bands, the system retains the *severity* information needed for compliance checking without revealing the exact count.

Evidence basis: ISO/IEC 20889:2018 Section 8.3 â€" range-based generalisation.

Band mapping for `affected_user_count` (in `_bucket_user_count()` in `field_mapper.py`):

Raw Count	Band	Connection
1â€"100	`minimal`	Below CERT-In thresholds
101â€"1,000	`moderate`	Standard reporting threshold
1,001â€"10,000	`large`	Elevated DPDP Board scrutiny
10,001+	`critical`	Maximum penalty territory

Technique 4 â€" Suppression

What it is: Simply removing columns entirely from the data before they enter the system.

Which columns are suppressed (defined in `DROP_COLUMNS` in `csv_loader.py`):

- `customer_master`: name, age, gender, email, phone, address â€" direct identifiers with zero compliance value
- `transaction_events`: account, device_id, ip_address â€" technical identifiers not needed for DPDP checks
- `data_lifecycle`: data_deleted, data_retained â€" redundant with retention_status
- `governance_config`: dpo_contact_masked â€" already masked, not needed in DB
- `policies`: policy_value_text â€" free text that could contain PII

The Gate Decision Framework

Every field in every table passes through a 3-gate decision:

Gate 1 â€" "Is this field needed to identify which tenant or record this belongs to?" If yes, keep it (e.g., `tenant_id`) or hash it (e.g., `customer_id` â†' `customer_hash`). If no, proceed to Gate 2.

Gate 2 â€" "Is this field needed by any of the 14 DPDP compliance rules?" If yes, keep it but apply appropriate anonymization technique (truncation, bucketing, ENUM validation). If no, proceed to Gate 3.

Gate 3 â€" "Does retaining this field provide calibration value for risk scoring?" Fields like `audit_frequency_days` and `consent_version` don't directly trigger violations but provide context. Gate 3 calibration (`gate3_calibration.py`) validates these fields stay within reasonable bounds.

Technique 5 â€" ENUM Validation

Why free text is dangerous: If a field like `consent_status` allowed any text, someone might enter `"John's consent was withdrawn on Tuesday"` â€" embedding PII in a supposedly safe field. ENUM validation forces every value into a predefined set.

How it works: `_validate_enum()` in `field_mapper.py` checks if the value is in the allowed list. If not, it logs a WARNING and maps to the safest default (e.g., `consent_status` defaults to `"expired"` â€" the most conservative option).

What happens when mapping fails: The field gets the default value, a warning is printed, and processing continues. The system never crashes on bad data â€" it degrades gracefully toward the most conservative compliance interpretation.

Processing Order: csv_loader â†' field_mapper â†' db_writer

This order is mandatory because:

1. **csv_loader** must run first to validate that all 10 files exist, filter by tenant_id, drop suppressed columns, and enforce schema constraints
2. **field_mapper** must run second because it needs clean DataFrames from csv_loader to apply hashing, truncation, bucketing, and ENUM validation
3. **db_writer** must run last because it needs the fully anonymized data and must insert tables in FK order

If you reversed field_mapper and db_writer, raw PII would enter the database. If you skipped csv_loader, you might process another tenant's data.

PART 4 â€" AGENT LAYER

Why Three Agents Instead of One Big Function?

The 14 DPDP rules are divided into three agents not by database table but by **legal obligation type**:

- **Regulation Agent** â€" checks rules about *consent and lawful processing* (rights of the Data Principal)
- **Audit Agent** â€" checks rules about *operational compliance* (duties of the Data Fiduciary)
- **Risk Agent** â€" checks rules about *security and third-party exposure* (technical safeguards)

This mirrors how real compliance teams work. In industry GRC platforms like OneTrust and RSA Archer, compliance checks are organized by obligation domain, not by data source. A single transaction record might be checked by multiple agents for different rules.

Evidence basis: ISO 31000 deterministic methodology â€" risk assessment should be structured by risk domain, with clear ownership and accountability per domain.

All three agents inherit from `BaseAgent` (`base_agent.py`), which provides shared methods: `_compute_rule_score()` (sums signal weights), `_classify_outcome()` (compares score to thresholds), and `_make_record()` (creates a `ViolationRecord`).

REGULATION AGENT (4 rules) â€" `regulation_agent.py`

This agent answers: "*Is this company processing data lawfully with proper consent?*"

DPDP-001: Invalid/Missing Consent Before Processing (Section 6(1)) Checks if any transaction was processed without valid consent. Queries `transaction_events` JOIN `consent_records`. Signals: expired/revoked consent (S1, 0.40), event after expiry (S2, 0.30), purpose mismatch (S3, 0.20), bundled consent (S4, 0.10). Severity: HIGH. Penalty: â‚¹150 crore.

DPDP-002: Processing Beyond Stated Purpose (Section 4(1)(b)) Checks if data was processed for a purpose different from what was consented. Even if consent is active, processing for a different purpose is illegal. Queries `transaction_events` JOIN `consent_records`. Signals: purpose mismatch (S1, 0.70), mismatch despite active consent (S2, 0.30). Severity: HIGH. Penalty: â‚¹150 crore.

DPDP-004: Missing Notice to Data Principal (Section 5) Checks if consent was collected without first providing a notice explaining what data would be processed and why. Queries `consent_records`. Signals: notice not provided (S1, 0.80), implicit/pre-ticked channel (S2, 0.20). Severity: MEDIUM. Penalty: â‚¹50 crore.

DPDP-005: Children's Data Without Guardian Consent (Section 9) The strictest rule. Checks if a minor's data is being processed without guardian consent. Queries `customer_master` LEFT JOIN `consent_records`. Signals: is_minor=true (S1, 0.50), no guardian_consent_hash (S2, 0.50). Severity: **CRITICAL**. Penalty: â‚¹200 crore. *Violation threshold is 0.40 (lower than most rules' 0.60) because even partial evidence of minors' data without guardian consent is actionable.*

Why these 4 belong together: They all concern the *legal basis for processing* â€" the core consent framework of the DPDP Act.

AUDIT AGENT (6 rules) â€" `audit_agent.py`

This agent answers: "*Is this company maintaining proper operational controls?*"

DPDP-003: Retention Beyond Allowed Period (Section 8(7)) Data retention means keeping personal data. The DPDP Act says you must delete it once the purpose is served. Signals: expiry passed (S1, 0.50), still active/expired status (S2, 0.30), purpose complete but not deleted (S3, 0.20). Queries `data_lifecycle`. Severity: HIGH.

DPDP-009: Missing Grievance Mechanism (Section 13) A grievance mechanism is a way for customers to complain about how their data is handled. Every company must have one. Signal: endpoint unavailable (S1, 1.00). Queries `governance_config`. Severity: LOW. Penalty: â‚¹10 crore.

DPDP-010: Failure to Honor Erasure Request (DSAR) (Section 12(a)) DSAR = Data Subject Access Request. When a customer asks for their data to be deleted, the company has 30 days. Signals: SLA breached (S1, 0.60), fulfillment still pending (S2, 0.40). Queries `dsar_requests`. Severity: HIGH.

DPDP-011: Excess Retention Without Purpose (Section 8(7)(a)) Similar to DPDP-003 but focused on purpose completion. If the purpose is done, the data must go. Signals: purpose complete (S1, 0.50), not deleted or pending deletion (S2, 0.50). Queries `data_lifecycle`. Severity: MEDIUM.

DPDP-012: Missing Audit Logs (Section 8(4)) Audit logs are a legal requirement â€" you must be able to prove what data was processed and when. Signals: audit frequency > 180 days (S1, 0.50), no last audit date (S2, 0.30), audit overdue (S3, 0.20). Queries `governance_config`. Severity: MEDIUM.

DPDP-013: Unauthorized Employee PII Access (Section 8(5)) Only 3 roles can view PII: `compliance_officer`, `underwriter`, and `data_analyst`. Anyone else accessing PII is a violation. Signals: accessed PII (S1, 0.40), unauthorized role (S2, 0.60). Queries `access_logs`. Severity: HIGH. Penalty: â‚¹250 crore.

Why these 6 belong together: They all concern *operational duties* â€" things the company must do as part of day-to-day compliance governance.

RISK AGENT (4 rules) â€" `risk_agent.py`

This agent answers: "*Is this company's data exposed to security and third-party risks?*"

DPDP-006: Unauthorized Third-Party Sharing (Section 6(1)) Sharing data with third parties without consent, or cross-border transfers. Signals: third-party sharing enabled (S1, 0.40), consent missing/inactive (S2, 0.40), cross-border transfer (S3, 0.20). Queries `transaction_events` LEFT JOIN `consent_records`. Severity: HIGH.

DPDP-007: Missing Data Processing Agreement (DPA) (Section 4(1)(c)) A DPA is a contract between the company and any third-party that processes data on its behalf. Without it, the company has no legal control over the third party. Signals: PII stored (S1, 0.50), third-party processor without DPA (S2, 0.50). Queries `system_inventory`. Severity: MEDIUM.

DPDP-008: Sensitive Data Without Encryption (Section 8(5)) Unencrypted PII is the most severe technical violation. Signals: pii_encrypted=false (S1, 0.70), encryption_type=none (S2, 0.30). Queries `security_events`. Severity: **CRITICAL**. Penalty: **â‚¹250 crore**.

DPDP-014: Delayed Breach Notification (Section 8(6)) The 72-hour window: once a breach is confirmed, the company must notify the Data Protection Board within 72 hours. Signals: breach detected (S1, 0.20), notification delay > 72h (S2, 0.50), large/critical affected users (S3, 0.30). Queries `security_events` where `breach_detected = true`. Severity: CRITICAL.

Why these 4 belong together: They all concern *external risk exposure* â€" third parties, encryption, and breach response.

PART 5 â€" WEIGHTED SIGNAL SCORING MODEL

What Is a "Signal"?

A signal is a single piece of evidence that suggests a violation might exist. Think of signals like symptoms in a medical diagnosis â€" no single symptom confirms a disease, but multiple symptoms together increase confidence.

Why Multiple Signals Instead of Binary Pass/Fail?

A binary approach would say: "consent expired? VIOLATION." But real compliance is nuanced. Maybe the consent just expired yesterday (less severe) versus expired a year ago with a purpose mismatch and bundled consent (very severe). The weighted signal model captures this nuance.

How Signal Weights Are Determined

Each signal's weight reflects its **diagnostic strength** â€" how strongly that evidence alone suggests a violation. Weights within each rule always sum to 1.0. The primary signal (strongest evidence) gets the highest weight.

Complete Example: DPDP-001 (Invalid Consent)

The rule has 4 signals defined in `regulation_agent.py`:

Signal	Description	Weight
S1	consent_status is expired/revoked/withdrawn	0.40
S2	event_date > expiry_date (processed after expiry)	0.30
S3	processing_purpose â‰ consented_purpose	0.20
S4	is_bundled = true (not specific consent)	0.10

How scoring works: `_compute_rule_score()` in `base_agent.py` sums the weights of all fired signals:

```
rule_score = Î£(weight_i)  for all signals that fired
```

Scenario A: Only S1 fires (consent expired) â†' score = 0.40 â†' **WARNING** (above 0.35 threshold, below 0.60)

Scenario B: S1 + S2 fire (expired AND processed after expiry) â†' score = 0.70 â†' **VIOLATION** (above 0.60 threshold)

Scenario C: No signals fire â†' score = 0.00 â†' **PASS**

Thresholds

Each rule has two thresholds set in `_classify_outcome()`:

- **Violation threshold:** Score at or above this = confirmed violation
- **Warning threshold:** Score at or above this (but below violation) = warning

Most rules use 0.60/0.35. But DPDP-005 (children's data) uses **0.40/0.30** because children's data is CRITICAL severity â€" even partial evidence is actionable.

Evidence Basis

- **Amaral et al. 2023 IEEE TSE** â€" demonstrated weighted evidence scoring for GDPR compliance achieves 89.1% precision
- **CVSS v3.1** â€" signal aggregation pattern for combining multiple vulnerability indicators into a single score

---

PART 6 — RISK SCORING MODULE

What the Formula Answers

"Out of all 14 DPDP obligations, how badly is this company failing — weighted by severity, volume, and likelihood of regulatory action?"

The risk score is a single number from 0 to 100 computed in `compute_risk_score()` in `orchestrator.py`.

The Likelihood x Impact Framework

Industry-standard from ISO 31000:2018 and NIST SP 800-30:

- **Likelihood** = How likely is the regulator to take action? (Based on prevalence and recurrence)
- **Impact** = How bad is the consequence? (Based on severity, financial penalty, signal strength)

Likelihood Component

Prevalence ratio: `prevalence_ratio = affected_records / total_relevant_records`

Why linear instead of logarithmic: The old system used `log(violations)` which dampened volume. With 276 violations, the old formula gave only 23.1/100. Linear ratio correctly reflects that 92% prevalence is near-total failure.

Recurrence factor (DPDP Act Section 33):

```
recurrence_factor = 1.0 + min(0.10 * (count - 1), 0.50)
```

First offense = 1.0, second = 1.10, maximum 1.50.

Final Likelihood: `L = prevalence_ratio * recurrence_factor`

Impact Component

Severity	Score
CRITICAL	1.00
HIGH	0.75
MEDIUM	0.50
LOW	0.25

Penalty weight: `penalty_weight = penalty_crore / 250`

Signal strength: `max(rule_score for all violations of this rule)`

Final Impact: `I = (0.4 * severity_score) + (0.3 * penalty_weight) + (0.3 * signal_strength)`

Per-Rule Inherent Risk and Aggregation

```
inherent_risk = L * I * risk_weight
raw_score = sum(inherent_risk) for all violated rules
max_possible = sum(risk_weight) for all 14 rules = 9.70
risk_score = min((raw_score / max_possible) * 100, 100)
```

Tier Allocation

Score Range	Tier
0	COMPLIANT
0.1 - 29.9	LOW
30.0 - 54.9	MEDIUM
55.0 - 74.9	HIGH
75.0 - 100	CRITICAL

Worked Example for TENANT-001 (score = 60.1)

DPDP-001 (Invalid Consent): 50 violations / 55 records, max score 0.70, HIGH, 150 Cr, weight 0.80

- L = (50/55) * 1.50 = 1.364, I = 0.69, inherent = 1.364 * 0.69 * 0.80 = **0.7529**

DPDP-008 (No Encryption): 10 violations / 12 records, CRITICAL, 250 Cr, weight 0.95

- L = (10/12) * 1.50 = 1.25, I = 1.00, inherent = 1.25 * 1.00 * 0.95 = **1.1875**

DPDP-009 (No Grievance): 1/1 records, LOW, 10 Cr, weight 0.30

- L = 1.0, I = 0.412, inherent = 1.0 * 0.412 * 0.30 = **0.1236**

Sum across ~10 violated rules = 5.83 -> (5.83 / 9.70) * 100 = **60.1** -> **HIGH** tier.

PART 7 — EXPLAINABILITY MODULE (XAI)

Why Explain Decisions?

A score alone is useless to a regulator. GDPR Article 22 establishes the right to explanation. DPDP Act Section 5 requires transparency. ISO 31000 requires repeatable, auditable methodology.

SHAP in Plain English

SHAP comes from game theory. Imagine signals as team players producing a score. SHAP asks: "How much did each player contribute?" For linear additive models, the SHAP value is exact:

```
phi_i = weight_i * fired_i    (1 if fired, 0 if not)
```

This is proven in Lundberg and Lee (2017) NeurIPS — not an approximation.

Walkthrough: DPDP-001 Explanation

Signal	Description	Weight	Fired	phi (SHAP)	Reason
S1	Consent expired/revoked	0.40	Yes	0.40	consent_status=expired
S2	Event after expiry	0.30	Yes	0.30	event_date > expiry_date
S3	Purpose mismatch	0.20	No	0.00	--
S4	Bundled consent	0.10	No	0.00	--
Total				**0.70**	

Root Cause Categories

Category	Meaning
PROCESS_GAP	No procedure exists
TECHNICAL_GAP	Procedure exists, controls missing
GOVERNANCE_GAP	Policy or oversight missing
IMPLEMENTATION_GAP	Procedure exists but not followed
HUMAN_ERROR	Unauthorized action by a person

Why Templates Instead of LLM Text

1. **Auditability:** Same rule always produces same explanation structure
2. **ISO 31000:** Methodology must be repeatable and deterministic
3. **No hallucination:** Templates contain only verified DPDP section references

---

PART 8 — REPORT GENERATION AND OUTPUT

The report is assembled by `report_builder.py`:

1. Executive Summary — One paragraph for CEO/board. States risk score, tier, violations, penalty exposure, top 3 rules.

2. Risk Assessment — Score (0-100), tier, financial exposure, rules violated vs. 14 checked.

3. Ranked Violations — Sorted by risk_contribution descending. Each includes SHAP table, root cause, legal citation, remediation.

4. Remediation Priority — Deduplicated by rule_id. Sorted: urgency (IMMEDIATE > HIGH > MEDIUM > LOW), then risk_contribution, then penalty. CRITICAL severity maps to IMMEDIATE urgency.

5. Agent Breakdown — Violations and warnings per agent.

6. PDF vs JSON — JSON feeds the React dashboard. PDF (`pdf_generator.py`, ReportLab) is for board presentations and regulatory filings. Includes color-coded severity badges, SHAP tables, and branded footer.

PART 9 — END-TO-END WALKTHROUGH

Follow a compliance officer at FinLend Technologies (TENANT-001):

Step 1: Officer opens React app, navigates to `Onboarding.jsx`. Uploads 10 CSVs, enters tenant_id and company name.

Step 2: API `/analyze/upload` (`routes.py`) receives files. `csv_loader.py` validates, filters by tenant, drops suppressed columns.

Step 3: `field_mapper.py` anonymizes: IDs become HMAC hashes (`hasher.py`), timestamps become dates, counts become bands, invalid ENUMs get safe defaults.

Step 4: `db_writer.py` inserts 10 tables in FK order within a single PostgreSQL transaction.

Step 5: `orchestrator.py` calls `run_compliance_analysis()`. Opens one DB connection for all agents.

Step 6: RegulationAgent (DPDP-001,002,004,005), AuditAgent (003,009,010,011,012,013), RiskAgent (006,007,008,014) each return `ViolationRecord` lists.

Step 7: `compute_risk_score()` groups violations by rule_id, queries DB for total record counts.

Step 8: For each rule: prevalence -> recurrence -> L; severity -> penalty -> signal -> I; L * I * weight = contribution.

Step 9: Sum / 9.70 * 100 = **60.1** -> tier = **HIGH**.

Step 10: `enrich_violations()` in `service.py` builds SHAP tables, assigns root causes, attaches remediation.

Step 11: `_build_full_response()` in `routes.py` assembles JSON with executive summary, remediation priority, agent breakdown.

Step 12: FastAPI returns JSON to React frontend.

Step 13: Dashboard renders: `ExecutiveSummary.jsx`, `MetricsSection.jsx`, `ChartsSection.jsx`, `ViolationsSection.jsx` (expandable SHAP cards), `RemediationSection.jsx`.

Step 14: Officer reads: "60.1/100, HIGH. 276 violations across 10 rules..."

Step 15: Expands DPDP-001 card, sees SHAP table: S1 (phi=0.40), S2 (phi=0.30), total 0.70.

Step 16: Reviews remediation: #1 DPDP-008 (IMMEDIATE, 250 Cr), #2 DPDP-013.

Step 17: Downloads PDF. `pdf_generator.py` queries `evaluation_results`, produces multi-page ReportLab PDF.

PART 10 — FUTURE SCOPE PER MODULE

Anonymization Module

- **Current:** 5 techniques at upload time via `field_mapper.py`
- **Future:** Automated Gate 3 calibration, differential privacy for aggregates, k-anonymity verification
- **Research:** Dwork and Roth (2014) "Algorithmic Foundations of Differential Privacy"

Agent Layer

- **Current:** 14 deterministic SQL-based rules across 3 agents
- **Future:** LLM-assisted edge-case interpretation, new rules as DPDP Rules are notified, natural language queries
- **Needs:** LLM API integration, rule versioning system

Risk Scoring

- **Current:** L x I with linear prevalence and recurrence escalation
- **Future:** Control effectiveness (residual risk = inherent * (1 - controls)), temporal decay for older violations
- **Research:** ISO 31000:2018 Annex A — control effectiveness assessment

Explainability

- **Current:** Template-based SHAP attribution in `service.py`
- **Future:** LLM narration layer, counterfactual explanations, multi-language reports
- **Research:** Wachter et al. (2017) "Counterfactual Explanations Without Opening the Black Box"

PDF Report

- **Current:** Static ReportLab PDF in `pdf_generator.py`
- **Future:** Interactive PDF with hyperlinks, DPDP Board submission format, automated DPO email

Frontend Dashboard

- **Current:** React/Vite with 6 sections
- **Future:** Historical trends, industry benchmarks, real-time webhooks, role-based views (DPO vs Board)

SUMMARY TABLE

Module	Technique	Evidence Basis	File(s)	Status	Future
Hashing	HMAC-SHA256	NIST SP 800-107 S5.3	`hasher.py`, `field_mapper.py`	Done	Differential privacy
Truncation	Date-only timestamps	ISO/IEC 20889 S8.3	`field_mapper.py`	Done	k-anonymity
Bucketing	Range bands	ISO/IEC 20889 S8.3	`field_mapper.py`	Done	Adaptive thresholds
Suppression	Column removal	Gate 1/2/3 framework	`csv_loader.py`	Done	Automated Gate 3
ENUM	Controlled vocabularies	ISO/IEC 20889	`field_mapper.py`	Done	Dynamic from config
Ingestion	CSV to PostgreSQL	FK-ordered writes	`csv_loader.py`, `db_writer.py`	Done	Streaming
Regulation Agent	4 consent rules	DPDP S4,5,6,9	`regulation_agent.py`	Done	LLM assist
Audit Agent	6 operational rules	DPDP S8,12,13	`audit_agent.py`	Done	New rules
Risk Agent	4 security rules	DPDP S4,6,8	`risk_agent.py`	Done	Control scoring
Signal Scoring	Weighted additive	Amaral 2023, CVSS v3.1	`base_agent.py`	Done	Bayesian updating
Risk Scoring	ISO 31000 LxI	ISO 31000, NIST 800-30	`orchestrator.py`	Done	Residual risk
Explainability	SHAP attribution	Lundberg and Lee 2017	`service.py`, `explanation.py`	Done	LLM narration
Report JSON	Structured API	--	`report_builder.py`, `routes.py`	Done	GraphQL
Report PDF	ReportLab	--	`pdf_generator.py`	Done	Interactive PDF
Frontend	React/Vite	--	`frontend/src/`	Done	Trends, RBAC

This document is self-contained. A reader with no prior context should now understand the complete Fin-Comply DPDP compliance system — from CSV upload to PDF report, from HMAC hashing to SHAP attribution, from signal weights to risk tiers.